Kernel Adaptive Memory

A Kernel-Transformer with Persistent Support Memory

Research working note

July 2026

Abstract

Kernel least-mean-square methods offer local, inspectable predictions and inexpensive online updates, but their geometry is usually fixed and their dictionaries can grow without bound. Transformers learn context-dependent routing, but their internal similarity geometry is less explicit and full self-attention scales quadratically with sequence length. This note develops a minimal bridge between these ideas. Sequence elements act as *temporary supports* for kernel self-attention; a finite learned bank supplies *persistent supports*; and a fast linear readout retains a KLMS-inspired adaptation path. The construction is intentionally conservative: one generic kernel-attention operator is used in two roles, the leading-order costs are stated explicitly, and four experiments isolate contextual retrieval, persistent prototypes, online dynamical adaptation, and natural-language prediction. The proposal is useful only if these components provide measurable value beyond matched dot-product and non-attention baselines.

1 Why this direction is relevant

The motivating tension is straightforward. KLMS predicts with an expansion of localized basis functions,

$$\widehat{y}(x) = \sum_i \alpha_i \kappa(x, x_i),$$

so a prediction can be decomposed into support-wise contributions and the coefficients admit simple online updates [1]. Its practical limitations are equally familiar: raw input distance may not match predictive similarity, support dictionaries tend to grow, and moving a support changes the meaning of previously learned coefficients.

Transformers solve a different problem. They learn which elements of the current sequence should interact, using query–key–value routing [2]. This is powerful for language and delayed dynamics, but the score is usually presented as an unconstrained dot product, learned global prototypes are not explicit, and dense self-attention over T tokens requires $O(T^2)$ pairwise weights. Learned input geometries and finite latent memories have close relatives in deep kernel learning, inducing-point attention, and latent-array models [3–5]; the proposed contribution is their fixed-budget, online-adaptive combination.

Central hypothesis

A useful middle ground may be obtained by treating attention as normalized kernel regression. Current sequence elements become temporary supports; a fixed-size learned bank provides persistent supports; and a fast readout adapts on top of the more slowly learned geometry. The three claims—kernel geometry, persistent memory, and online adaptation—must be tested separately before their combination is credited.

This idea is plausible for four reasons. First, a normalized positive kernel already has the algebraic form of attention. Second, Gaussian attention becomes scaled dot-product attention when query and key norms are controlled. Third, self-attention is obtained simply by allowing the current tokens to serve as one another's supports. Fourth, a final predictor can remain linear in selected fast parameters even when the context geometry is learned nonlinearly.

2 One operator, two kinds of memory

2.1 Kernel attention in one line

Let $Q \in \mathbb{R}^{T_q \times r}$, $K \in \mathbb{R}^{T_k \times r}$, and $V \in \mathbb{R}^{T_k \times d_v}$. Define the pairwise learned distance

$$\Delta_D(Q,K)_{ij} = (q_i - k_j)^\top D(q_i - k_j), \qquad D \succeq 0, \tag{1}$$

and the generic kernel-attention operator

$$\mathrm{KAttn}(Q,K,V;B) = \mathrm{softmax}_{\mathrm{row}}\left(-\frac{\Delta_D(Q,K)}{2\sigma^2} + B\right)V. \tag{2}$$

Here B contains a causal mask, a relative-position bias, or zero. Each row of the softmax is a normalized Gaussian kernel over the available keys. Lower energy means greater influence.

All attention operators below are multihead. The head index is omitted for readability: each head has its own projections, positive metric, and bandwidth; head outputs are concatenated and projected. In the first implementation, D is diagonal and positive and each head has one bandwidth. This avoids the $O(r^2)$ pair cost of a dense metric.

2.2 Why the score remains transformer-like

For $D = I$,

$$-\frac{\|q-k\|^2}{2\sigma^2} = \frac{q^\top k}{\sigma^2} - \frac{\|q\|^2}{2\sigma^2} - \frac{\|k\|^2}{2\sigma^2}. \tag{3}$$

For a fixed query, the query-norm term cancels in the softmax. If keys have equal norm, the key-norm term also cancels, leaving scaled dot-product attention with temperature σ^2. Thus radial attention is not an unrelated mechanism; it is an explicit energy parameterization of the same normalized routing operation. Asymmetric query and key maps remain allowed. They make the attention score useful even when observed inputs and learned supports play different roles, although the resulting raw-input similarity need not be a symmetric Mercer kernel.

2.3 Context memory: tokens as temporary supports

Given token representations $H = [h_1, \ldots, h_T]^\top$, define

$$\mathrm{Ctx}(H) = \mathrm{KAttn}\left(HW_Q^c, HW_K^c, HW_V^c; B_{\mathrm{causal}}\right). \tag{4}$$

The causal mask allows token t to query only permitted earlier positions. This is self-attention because the queries, keys, and values all come from the same current sequence. Every earlier token acts as a temporary support whose location and value depend on the current example.

2.4 Persistent memory: a finite learned support bank

Let

$$\mathcal{B} = (K_p, V_p), \qquad K_p \in \mathbb{R}^{M \times r}, \quad V_p \in \mathbb{R}^{M \times d_v},$$

be a learned bank of M support keys and values. Define

$$\mathrm{Mem}(U;\mathcal{B}) = \mathrm{KAttn}\left(UW_Q^m, K_p, V_p; 0\right). \tag{5}$$

This is cross-attention: contextual tokens query parameters that persist across examples. The bank is intended to represent reusable regimes, prototypes, or local predictive behaviors. Its size M is fixed, so model memory does not grow with the data stream.

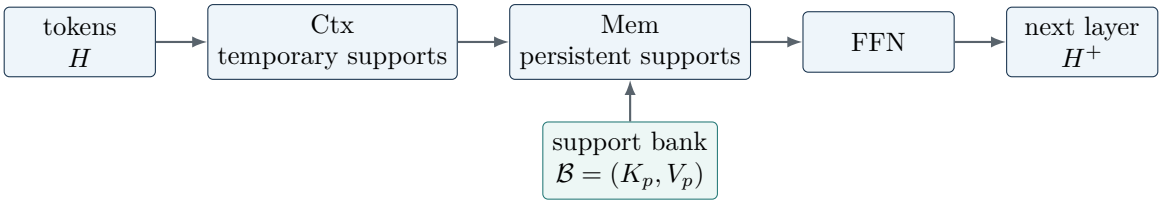

Figure 1: One block uses the same kernel-attention operator twice. Context attention retrieves observations from the current sequence; memory attention retrieves reusable learned supports. Residual paths and normalization are shown in equation (6).

2.5 The complete block

With pre-normalization, residual connections, and a position-wise map,

$$\begin{aligned} U &= H + \mathrm{Ctx}(\mathrm{LN}(H)), \\ Z &= U + \mathrm{Mem}(\mathrm{LN}(U); \mathcal{B}), \\ H^+ &= Z + \mathrm{FFN}(\mathrm{LN}(Z)). \end{aligned} \tag{6}$$

Stacking this block produces a transformer. The defining transformer component is Ctx, which constructs a token-to-token $T \times T$ interaction matrix. The persistent bank is a distinctive augmentation, not a replacement for self-attention. Support-to-support attention is omitted initially because normalized routing already creates competition among supports and direct support interactions would weaken their interpretation as stable centers.

Implementation contract

The reference model uses normalized query/key vectors for radial attention, a positive diagonal metric, one bandwidth per head, separate softmax normalizations for Ctx and Mem, a fixed support count M, and no support birth/death. Geometry is trained by backpropagation. Only the final scalar readout receives the optional NLMS update.

3 Fast adaptation without moving the full model

For scalar prediction, let z_t denote the final contextual token and let $a_t^p \in \mathbb{R}^M$ be its persistent-support routing vector, averaged or concatenated across heads. Define

$$\psi_t = \begin{bmatrix} z_t \\ a_t^p \\ 1 \end{bmatrix}, \qquad \widehat{y}_t = \theta_t^\top \psi_t. \tag{7}$$

When the attention geometry is fixed, squared-error learning in θ is convex. A normalized-LMS update is

$$\theta_{t+1} = \theta_t + \eta \, \frac{(y_t - \widehat{y}_t)\psi_t}{\epsilon + \|\psi_t\|^2}. \tag{8}$$

This creates two timescales: the readout can adapt after every observation, while query/key maps, metrics, bandwidths, and persistent supports move more slowly. If the geometry later moves substantially, the old function should be transported or refit on an anchor set before claiming stable continual learning. For language modeling, the final layer is instead a vocabulary softmax trained by a cross-entropy gradient; the LMS claim does not apply directly.

4 Efficiency and timing at a glance

Let T be sequence length, W a local context window, M the persistent support count, d the model width, and d_{ff} the feed-forward width. With a fixed number of heads whose dimensions sum to d, the leading costs are:

Table 1: Leading-order cost per block, omitting batch size and constant factors.

Component	**Time**	**Stored attention / state**
Full context attention	$O(T^2 d)$	$O(T^2)$
Local context attention	$O(TWd)$	$O(TW)$
Persistent memory attention	$O(TMd)$	$O(TM) + O(Md)$ parameters
Feed-forward map	$O(Tdd_{ff})$	$O(Td_{ff})$ activations
Incremental token, local cache	$O((W+M)d + dd_{ff})$	$O((W+M)d)$ cache/state

A diagonal radial metric can be evaluated as a transformed matrix product plus query/key norm terms. It therefore has the same asymptotic pairwise cost as dot-product attention, but a larger constant factor. A dense metric would increase pairwise work and is not justified in the first study. Local context attention changes the hybrid attention cost from

$$O(T^2 d + TMd) \quad \text{to} \quad O(TWd + TMd).$$

Big-O notation is not a substitute for timing. Every comparison should report forward latency, backward latency, tokens or samples per second, peak memory, parameter count, precision, hardware, and warm-up protocol. The companion benchmark synchronizes GPU measurements and sweeps T while holding width, depth, batch size, and support count fixed. Linearized kernel attention is a later option when exact pairwise attribution is no longer required [6].

5 A falsifiable experimental ladder

The companion `kam-experiments` codebase implements four sequence "languages." Each one isolates a different architectural claim before the model is scaled.

Table 2: Experiments ordered from mechanism tests to realistic stress tests.

Language	**Scientific question**	**Primary comparison**	**Main measurements**
Copy language	Can context attention retrieve an exact earlier symbol?	radial Ctx vs dot-product self-attention	copied-token accuracy, length generalization, deletion tests
Hidden-regime grammar	Does the persistent bank discover reusable transition rules?	hybrid vs context-only	next-token loss, support/regime purity, support utilization
Mackey–Glass	Can the model use delayed context and adapt after a dynamical shift?	hybrid vs GRU, MLP, budgeted KLMS	one-step MSE, rollout error, post-shift recovery, NLMS gain
Character text	Is the method competitive on a recognizable language task?	hybrid vs matched dot transformer	perplexity, throughput, memory, attention diagnostics

The core ablation grid is:

1. **kernel-self**: Ctx only;
2. **memory-only**: Mem only;
3. **KAM**: radial Ctx + Mem;

4. **dot-transformer**: matched dot-product Ctx only;
5. **dot-hybrid**: matched dot-product Ctx + Mem;
6. task-appropriate GRU, MLP, or fixed-budget KLMS controls.

All comparisons should match model width, depth, head count, optimizer, training tokens, and evaluation budget; report both parameter count and measured wall-clock time. Interpretability is not established by attractive heatmaps. High-weight tokens and supports must survive perturbation tests: deleting or masking an attributed element should change the prediction in the expected direction. Persistent supports should also remain used, separated, and reasonably stable across seeds.

Decision rule

Continue only if the hybrid demonstrates at least one defensible advantage at matched cost: better prediction, faster post-shift adaptation, lower persistent memory, or causally validated explanations. If radial attention is slower and no more accurate or interpretable than the dot-product control, retain the simpler transformer and reject the radial component. If the persistent bank is unused or redundant after context attention, remove it.

6 Next steps if the minimal method works

1. **Reproducibility first.** Run the full ablation grid on three or more seeds; publish commands, checkpoints, timing tables, and raw attention diagnostics.
2. **Validate the geometry.** Measure support usage, effective rank, bandwidths, routing entropy, nearest contexts, perturbation faithfulness, and stability across seeds.
3. **Add controlled plasticity.** Permit only a subset of persistent supports to move or be replaced; introduce support repulsion and function-preserving coefficient transport.
4. **Scale the sequence tests.** Move from character text to subword language, longer contexts, local or sparse context attention, and larger fixed banks.
5. **Test scientific sequences.** Apply the same separation—temporary observed states versus persistent regimes—to Burgers-type dynamics, latent PDE rollouts, and patchwise flow prediction. Evaluate rollout stability and physical quantities, not only one-step error.
6. **Optimize the implementation.** Compare diagonal, low-rank, and compactly supported scores; profile fused distance kernels; and determine whether sparse routing provides a real systems advantage.
7. **Develop theory around the observed regime.** Only after the useful configuration is known, study approximation error, two-timescale stability, support transport, and dynamic regret under distribution shift.

7 Questions that should remain open

1. Is the radial energy itself useful, or is the persistent support bank the only valuable addition?
2. Does the bank learn global regimes that generalize across sequences, or merely duplicate the feed-forward layer?
3. Which task dimension controls performance: raw dimension, kernel dimension, context length, or support count?
4. Can support meaning remain stable while the model adapts online?
5. At what context and support sizes does local or sparse routing become necessary in practice?

8 Conclusion

The proposed model is a transformer for a precise reason: its current tokens perform causal query–key–value interaction through Ctx. The kernel contribution is the radial energy used to score those interactions. The persistent bank adds bounded cross-attention memory through Mem. The KLMS contribution is a fast linear adaptation path on top of the learned representation. This decomposition makes the proposal testable: each claim has a matched control, a measurable cost, and a clear condition under which it should be discarded.

References

[1] Weifeng Liu, Puskal P. Pokharel, and Jose C. Principe. The kernel least-mean-square algorithm. *IEEE Transactions on Signal Processing*, 56(2):543–554, 2008. doi: 10.1109/TSP.2007.907881.

[2] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[3] Andrew Gordon Wilson, Zhiting Hu, Ruslan Salakhutdinov, and Eric P. Xing. Deep kernel learning. In *Proceedings of the 19th International Conference on Artificial Intelligence and Statistics*, volume 51 of *Proceedings of Machine Learning Research*, pages 370–378. PMLR, 2016. URL https://proceedings.mlr.press/v51/wilson16.html.

[4] Juho Lee, Yoonho Lee, Jungtaek Kim, Adam Kosiorek, Seungjin Choi, and Yee Whye Teh. Set transformer: A framework for attention-based permutation-invariant neural networks. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 3744–3753. PMLR, 2019. URL https://proceedings.mlr.press/v97/lee19d.html.

[5] Andrew Jaegle, Felix Gimeno, Andrew Brock, Andrew Zisserman, Oriol Vinyals, and Joao Carreira. Perceiver: General perception with iterative attention. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 4651–4664. PMLR, 2021. URL https://proceedings.mlr.press/v139/jaegle21a.html.

[6] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and Francois Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 5156–5165. PMLR, 2020. URL https://proceedings.mlr.press/v119/katharopoulos20a.html.